Clase 12 (Unidad 5) — Testing, CI/CD y despliegue en producción

Duración total: 4 horas | Proyecto: tests automáticos + pipeline + despliegue de TaskFlow

Cómo leer esta guía

Cada parte combina dos tipos de contenido:

- **Conceptos** (qué es y por qué): la base teórica de la Unidad 5, con tablas y diagramas. Se pueden leer antes de la clase.
- **Práctica** (cómo): los pasos con código, que aplican esos conceptos a TaskFlow.

Al final hay **preguntas de repaso** para autoevaluarse.

Parte 0 — Antes de empezar (5 min)

Parados en `taskflow/`, entorno virtual activo. Última clase: vamos a cerrar el ciclo completo, desde tests automáticos hasta un despliegue real accesible por internet.

Parte 1 — Testing automatizado (70 min)

1.1 Por qué testear

Hasta ahora verificamos todo a mano, con `curl`, Postman o la Browsable API, clase a clase. Eso no escala: en algún momento van a cambiar algo de la Clase 5 (por ejemplo, la capa de servicios) y romper, sin darse cuenta, algo de la Clase 7 (los permisos por rol). Los **tests automáticos** son la red de seguridad que detecta esas roturas antes de que lleguen a producción.

Concepto: la pirámide de tests. No todos los tests son iguales. La forma clásica de organizarlos es una pirámide: muchos tests chicos y rápidos en la base, y pocos tests grandes y lentos arriba.

```
          /\
         /  \        Punta a punta (end-to-end): pocos
        / E2E\       el sistema completo, como lo usa una persona
       /------\
      /        \     Integración: bastantes
     / Integr.  \    varias piezas juntas (vista + permisos + base)
    /------------\
   /              \   Unitarios: muchos
  /   Unitarios    \  una función o clase, aislada
 /------------------\
```

Tipo	Qué prueba	En TaskFlow	Velocidad	Si falla, indica...
Unitario	Una pieza aislada	`TaskService.mark_completed`	Milisegundos	Exactamente qué función está mal
Integración	Varias piezas juntas	`POST /api/tasks/` con permisos y base de datos	Décimas de segundo	Que algo falla en la combinación
Punta a punta (*end-to-end*, E2E)	El sistema completo desde afuera	Un navegador automático que abre el frontend (Clase 13), se loguea y crea una tarea	Segundos	Que el usuario no puede hacer algo, pero no dónde está el error

Las pruebas E2E se escriben con herramientas que manejan un navegador real, como **Playwright** o **Selenium**. Son las más realistas y las más frágiles (cualquier cambio en la pantalla las rompe), por eso son pocas.

Concepto: cómo se escribe un buen test.

- **Patrón AAA** (*Arrange, Act, Assert*): **preparar** el escenario, **ejecutar** lo que se prueba y **verificar** el resultado. Los tests de esta clase están escritos así, separados por líneas en blanco.

- **Principios FIRST:** rápidos (*Fast*), independientes entre sí (*Independent*), repetibles con el mismo resultado en cualquier máquina (*Repeatable*), que se verifiquen solos, sin mirar la salida a mano (*Self-validating*), y escritos junto con el código, no meses después (*Timely*).
- **Un test por comportamiento,** con un nombre que lo describa: `test_miembro_no_puede_crear_tarea` dice qué se espera sin leer el código.
- **Probar también lo que no debe pasar:** los errores, los permisos denegados, los datos inválidos.

Concepto: dobles de prueba. Cuando una pieza depende de algo lento, externo o impredecible (una API, un modelo de IA, Redis), se la reemplaza durante el test por un **doble**:

Doble	Qué hace	Ejemplo
Stub	Devuelve respuestas fijas	La lista de feriados falsa del `conftest.py`
Mock	Además, verifica cómo fue llamado	Comprobar que se llamó a la IA con el objetivo correcto
Fake	Una implementación simple pero funcional	`InMemoryChannelLayer` y `LocMemCache` en lugar de Redis

En el día a día se les dice "mocks" a todos, pero conocer la diferencia ayuda a elegir. Una práctica relacionada es el **TDD** (*desarrollo guiado por tests*): escribir el test **antes** que el código, verlo fallar, y recién ahí programar lo necesario para que pase.

1.2 Instalación de pytest-django

Parados en `taskflow/`, entorno virtual activo:

```
pip install pytest pytest-django pytest-cov
pip freeze > requirements.txt
```

`pytest` es el ejecutor de tests más usado en Python; `pytest-django` lo conecta con Django (base de datos de prueba, settings); `pytest-cov` mide la cobertura (Parte 1.6).

Creen el archivo `taskflow/pytest.ini` (en la raíz, junto a `manage.py`):

```
[pytest]
DJANGO_SETTINGS_MODULE = config.settings
python_files = tests.py test_*.py *_tests.py
filterwarnings =
    ignore:No directory at:UserWarning
```

- `DJANGO_SETTINGS_MODULE` le indica a `pytest` qué configuración de Django cargar antes de correr cualquier test (sin esto, no sabría dónde están los modelos ni la base de datos).
- `python_files` define qué patrones de nombre de archivo reconoce `pytest` como archivos de test.
- `filterwarnings` silencia un aviso inofensivo de WhiteNoise (lo instalamos en la Parte 3) cuando todavía no se generaron los archivos estáticos.

1.3 La carpeta de tests

Django creó en la Clase 1 un archivo `core/tests.py` vacío. Lo vamos a reemplazar por una **carpeta** `core/tests/`, para poder separar los tests en varios archivos. Los dos no pueden convivir: Python no sabría si `core.tests` es el archivo o la carpeta, y `pytest` se corta con un error de import antes de correr nada. Parados en `taskflow/`:

```
git rm core/tests.py
mkdir -p core/tests
touch core/tests/__init__.py
```

`git rm` borra el archivo y le avisa a Git que se eliminó. `__init__.py` (vacío) convierte la carpeta en un paquete de Python. En Windows PowerShell, en vez de `mkdir -p` y `touch`: `mkdir core\tests` y `New-Item core\tests\__init__.py`.

1.4 Fixtures: preparar el escenario de cada test

Concepto: aislamiento. Cada test tiene que poder correr solo, en cualquier orden, y dar siempre el mismo resultado. Si un test depende de lo que dejó otro, un día fallan "sin motivo" y nadie sabe por qué. `pytest-django` lo garantiza para la base de datos: cada test corre dentro de una transacción que se **deshace al terminar**, así que los datos que crea un test no existen para el siguiente. Las fixtures son la otra mitad: cada test declara qué necesita y lo recibe recién creado.

Práctica.

Muchos tests necesitan lo mismo: un par de usuarios, un proyecto con membresías, un cliente HTTP. En vez de repetirlo en cada test, `pytest` usa **fixtures**: funciones que preparan algo y se lo entregan a los tests que lo pidan por nombre.

Creen `taskflow/conftest.py` (en la raíz, junto a `manage.py`; `pytest` lo carga automáticamente):

```
import pytest
from django.contrib.auth.models import User
from rest_framework.test import APIClient

from core.models import Project, ProjectMembership
from core.services import HolidayService


@pytest.fixture(autouse=True)
def entorno_de_test(settings, monkeypatch):
    # Sin Redis: el channel layer vive en memoria durante los tests
    settings.CHANNEL_LAYERS = {"default": {"BACKEND": "channels.layers.InMemoryChannelLayer"}}
    # Sin internet: la API de feriados devuelve una lista fija
    monkeypatch.setattr(HolidayService, "get_holidays", classmethod(lambda cls, year: [f"{year}-05-25"]))
    # Sin límites de peticiones
    settings.REST_FRAMEWORK = {**settings.REST_FRAMEWORK, "DEFAULT_THROTTLE_CLASSES": []}


@pytest.fixture
def ana(db):
    return User.objects.create_user(username="ana", password="clave12345")


@pytest.fixture
def beto(db):
    return User.objects.create_user(username="beto", password="clave12345")


@pytest.fixture
def proyecto(ana, beto):
    project = Project.objects.create(name="Proyecto de Ana")
    ProjectMembership.objects.create(project=project, user=ana, role="admin")
    ProjectMembership.objects.create(project=project, user=beto, role="miembro")
    return project


@pytest.fixture
def cliente():
    return APIClient()
```

- `@pytest.fixture(autouse=True)` se aplica **automáticamente** a todos los tests. Prepara un entorno aislado del mundo exterior: los tests no pueden depender de que Redis esté prendido ni de que haya internet, porque en la máquina de GitHub Actions (Parte 2) no hay Redis, y la API de feriados podría estar caída justo ese día.
- `settings` es una fixture de `pytest-django` que permite cambiar la configuración **solo durante el test**; al terminar, vuelve a su valor original.
- `monkeypatch.setattr(...)` reemplaza temporalmente `HolidayService.get_holidays` por una función falsa que siempre devuelve el 25 de mayo. Es la técnica de **mock**: sustituir una dependencia externa por un doble controlado.
- `{**settings.REST_FRAMEWORK, "DEFAULT_THROTTLE_CLASSES": []}` copia el diccionario de la Clase 8 y pisa solo esa clave: sin eso, un test que haga muchas peticiones podría recibir un `429`.

- `ana(db)`: pedir la fixture `db` (de `pytest-django`) habilita el acceso a la base de datos. `proyecto(ana, beto)` depende de otras dos fixtures: `pytest` las crea primero y se las pasa.

1.5 Tests unitarios (Service) y de integración (endpoints)

Creen `taskflow/core/tests/test_services.py`:

```
from datetime import date

import pytest
from rest_framework.exceptions import ValidationError

from core.models import Project, Task
from core.services import HolidayService, TaskService


@pytest.mark.django_db
def test_mark_completed_cambia_el_status():
    project = Project.objects.create(name="Demo")
    task = Task.objects.create(project=project, title="Test", due_date=date(2030, 1, 10))

    TaskService.mark_completed(task)
    task.refresh_from_db()

    assert task.status == "completada"


@pytest.mark.django_db
def test_mark_completed_sin_fecha_limite_falla():
    project = Project.objects.create(name="Demo")
    task = Task.objects.create(project=project, title="Sin fecha")

    with pytest.raises(ValidationError):
        TaskService.mark_completed(task)


def test_is_holiday_usa_la_lista_de_feriados():
    assert HolidayService.is_holiday(date(2030, 5, 25)) is True
    assert HolidayService.is_holiday(date(2030, 5, 26)) is False
```

- `@pytest.mark.django_db` es obligatorio en cualquier test que toque la base de datos. `pytest-django` crea automáticamente una base de datos **de prueba**, separada de `db.sqlite3`, la usa durante el test y la descarta al final: los tests nunca ensucian ni dependen de los datos que cargaron a mano en clases anteriores.
- `task.refresh_from_db()` vuelve a leer el objeto desde la base de datos, para verificar lo que realmente quedó guardado y no lo que quedó en memoria.
- `assert` es la instrucción de Python que, si la condición es falsa, hace fallar el test con un mensaje claro.
- `with pytest.raises(ValidationError):` verifica que el código de adentro **lance** esa excepción. Es la regla de negocio de la Clase 5: no se completa una tarea sin fecha límite. Testear que los errores ocurren cuando deben es tan importante como testear el camino feliz.
- El tercer test no usa la base de datos (no lleva el marcador): usa la lista falsa del `conftest.py`.

Creen `taskflow/core/tests/test_views.py`:

```
import pytest

from core.models import Project, Task


@pytest.mark.django_db
def test_listar_tareas_requiere_autenticacion(cliente):
    response = cliente.get("/api/tasks/")
    assert response.status_code == 401
```

```
@pytest.mark.django_db
def test_usuario_solo_ve_tareas_de_sus_proyectos(cliente, ana, proyecto):
    Task.objects.create(project=proyecto, title="Mía")
    otro = Project.objects.create(name="Proyecto ajeno")
    Task.objects.create(project=otro, title="Ajena")

    cliente.force_authenticate(user=ana)
    response = cliente.get("/api/tasks/")

    assert response.status_code == 200
    titulos = [t["title"] for t in response.data["results"]]
    assert titulos == ["Mía"]


@pytest.mark.django_db
def test_admin_puede_crear_tarea(cliente, ana, proyecto):
    cliente.force_authenticate(user=ana)
    response = cliente.post("/api/tasks/", {"project": proyecto.id, "title": "Nueva"}, format="json")
    assert response.status_code == 201


@pytest.mark.django_db
def test_miembro_no_puede_crear_tarea(cliente, beto, proyecto):
    cliente.force_authenticate(user=beto)
    response = cliente.post("/api/tasks/", {"project": proyecto.id, "title": "Nueva"}, format="json")
    assert response.status_code == 403
```

- `APIClient` es un cliente HTTP de prueba, provisto por DRF, que simula peticiones sin necesidad de un servidor corriendo de verdad.
- `client.force_authenticate(user=user)` es un atajo de testing que simula "este request ya está autenticado como este usuario", sin generar un JWT real para cada test.
- `response.data["results"]`: la respuesta viene **paginada** (Clase 5), por eso las tareas están dentro de `results`.
- El segundo test es el importante: no alcanza con verificar que responde `200`, hay que verificar que **no aparece** la tarea ajena. Es el test del *Broken Access Control* de la Clase 8.
- Los dos últimos tests cubren la regla de roles de la Clase 7, incluido el caso del `POST`, el que `has_object_permission` no cubre.

1.6 Ejecutar y medir cobertura

Parados en `taskflow/`:

```
pytest
```

Deberían ver `7 passed`. Para ver el nombre de cada test: `pytest -v`. Para medir **cobertura** (qué porcentaje de las líneas de código se ejecutaron durante algún test):

```
pytest --cov=core
```

La tabla muestra, archivo por archivo, cuántas líneas hay (`Stmts`), cuántas no se ejecutaron nunca (`Miss`) y el porcentaje (`Cover`). Para ver exactamente **cuáles** líneas faltan: `pytest --cov=core --cov-report=term-missing`.

> 💡 **Nota del docente**
> Un número de cobertura alto no garantiza ausencia de bugs (se puede "ejecutar" una línea sin verificar realmente que hace lo correcto), pero un número muy bajo sí garantiza zonas del código sin ninguna verificación automática. Para esta materia, apunten a cubrir al menos las reglas de negocio (`core/services.py`) y los permisos (`core/permissions.py`).

Concepto: qué mide la cobertura. La **cobertura de líneas** (la que vimos) dice qué líneas se ejecutaron. La **cobertura de ramas** (`pytest --cov=core --cov-branch`) es más exigente: en un `if`, verifica que se hayan probado **los dos**

caminos, el que entra y el que no. Un `if` con un solo camino probado tiene sus líneas "cubiertas", pero la mitad de su comportamiento sin verificar.

Hagan el experimento que justifica todo esto: en `core/views.py`, comenten temporalmente la línea `self.check_project_admin(...)` de `perform_create` y vuelvan a correr `pytest`. El test `test_miembro_no_puede_crear_tarea` falla y les muestra exactamente qué regla se rompió. Descomenten la línea.

Parte 2 — Integración continua (CI) con GitHub Actions (55 min)

2.1 Concepto: integración continua y entrega continua

CI (Integración Continua) significa: cada vez que alguien sube código a GitHub, un proceso **automático** (no una persona revisando a mano) corre los tests del proyecto. Si fallan, el equipo se entera inmediatamente, antes de que ese código llegue a producción. Retomando lo visto en el documento de Git y GitHub, esto ocurre disparado por cada `git push`.

La integración continua es, sobre todo, una **forma de trabajar** en equipo, que la herramienta hace posible:

- Integrar cambios **chicos y frecuentes** a la rama principal, en vez de pasar semanas en una rama aparte y "fusionar todo" al final.
- La rama `main` tiene que estar **siempre en verde** (con los tests pasando). Si se rompe, arreglarla es la prioridad del equipo.
- Cada cambio entra por un *pull request*, que el pipeline verifica **antes** de aceptarlo. En GitHub se puede exigir con **reglas de protección de rama**: no se puede fusionar si el pipeline falló.

La "CD" de CI/CD tiene dos significados, y conviene distinguirlos:

Práctica	Qué se automatiza	Quién decide salir a producción
Entrega continua (*Continuous Delivery*)	Todo, hasta dejar una versión **lista** para publicar	Una persona, con un botón
Despliegue continuo (*Continuous Deployment*)	Todo, incluida la publicación	Nadie: cada cambio que pasa los tests sale solo

Vocabulario de GitHub Actions, para leer cualquier pipeline:

Término	Qué es	En nuestro `ci.yml`
Workflow	El archivo con todo el proceso	`ci.yml`
Evento	Lo que lo dispara	`push` y `pull_request` a `main`
Job	Un conjunto de pasos que corre en una máquina	`test`
Runner	La máquina virtual que ejecuta el job	`ubuntu-latest`
Step	Un paso: un comando o una acción	"Instalar dependencias"
Action	Un paso reutilizable publicado por otros	`actions/checkout@v4`
Secret	Una variable cifrada del repositorio	(en la clase de despliegue)

2.2 El pipeline

Parados en `taskflow/`, creen la estructura de carpetas exacta que GitHub Actions espera (los nombres de carpeta son obligatorios, no elegidos por nosotros):

```
mkdir -p .github/workflows
```

Creen el archivo `taskflow/.github/workflows/ci.yml`:

```
name: CI

on:
  push:
```

```
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    env:
      SECRET_KEY: clave-solo-para-los-tests-de-ci-0123456789
      DEBUG: "False"
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Instalar dependencias
        run: pip install -r requirements.txt
      - name: Ejecutar tests
        run: pytest --cov=core
```

💡 **Nota del docente**
Este archivo está escrito en **YAML**, un formato de configuración estándar de la industria (no es código Python) que usa GitHub Actions para describir el pipeline. Es una de las excepciones al "todo Python/Django" señaladas desde la Clase 1: es inevitable porque es el lenguaje que exige esta herramienta, no una elección nuestra. En YAML **la sangría importa** (como en Python) y se usan espacios, nunca tabulaciones.

Desglosando:

- `on: push/pull_request` define **cuándo** se dispara el pipeline: en cada push a `main` y en cada *pull request* hacia `main`.
- `runs-on: ubuntu-latest` es la máquina virtual (provista gratis por GitHub) donde va a correr.
- `env:` define variables de entorno para todos los pasos. Es **imprescindible**: el `.env` no se sube al repositorio (Clase 1), así que en la máquina de GitHub no existe, y sin `SECRET_KEY` Django no puede arrancar (los tests fallan con `ImproperlyConfigured: The SECRET_KEY setting must not be empty`). Usamos una clave inventada, válida solo para los tests: **nunca** pongan acá la clave real de producción.
- Cada `step` es un paso secuencial: `actions/checkout@v4` descarga el código del repositorio en esa máquina virtual, `actions/setup-python@v5` instala Python 3.12, y los dos últimos pasos instalan las dependencias (con el mismo `requirements.txt` que venimos manteniendo desde la Clase 1) y corren los tests.
- No hace falta Redis ni internet: el `conftest.py` de la Parte 1.4 ya los reemplaza.

Guarden el archivo y súbanlo con el flujo habitual de Git (Parte 4 de esta misma clase). En la pestaña **Actions** del repositorio en github.com van a ver el pipeline ejecutándose automáticamente apenas hagan `git push`: una tilde verde si pasó, una cruz roja si algo falló (hagan clic para ver el detalle).

💡 **Nota del docente**
Si en la clase extra de MySQL instalaron `mysqlclient`, el paso "Instalar dependencias" puede fallar en GitHub por falta de librerías del sistema. Agreguen antes un paso: `- run: sudo apt-get update && sudo apt-get install -y pkg-config default-libmysqlclient-dev`.

Parte 3 — Contenedor Docker y despliegue en la nube (80 min)

3.0 Instalar Docker

Docker es un programa que empaqueta una aplicación junto con todo lo necesario para que corra igual en cualquier computadora. **Docker Desktop** (la aplicación con interfaz gráfica que instala también el motor de Docker) es gratuita para uso personal, educativo y en empresas pequeñas (menos de 250 empleados o menos de 10 millones de dólares de facturación anual), que es exactamente nuestro caso; empresas grandes necesitan una licencia paga, algo que no nos afecta acá.

- **Windows:** descarguen Docker Desktop desde `docker.com/products/docker-desktop`. Requiere WSL2 habilitado (si siguieron la Opción C de Redis en la Clase 10, ya lo tienen; si no, el instalador de Docker Desktop ofrece activarlo automáticamente).
- **Mac:** mismo instalador desde `docker.com/products/docker-desktop`, eligiendo la versión para Apple Silicon o Intel según corresponda.
- **Linux:** instalen el motor de Docker siguiendo la guía oficial para su distribución en `docs.docker.com/engine/install/`. Instalen también el plugin **Compose** (en Ubuntu, el paquete `docker-compose-plugin` de la guía oficial), que usamos en la Parte 3.3.

Verifiquen la instalación, en cualquier sistema operativo:

```
docker --version
docker compose version
```

> 💡 **Nota del docente**
> Si en algún paso de esta clase `docker` no es reconocido como comando, revisen que Docker Desktop (Windows/Mac) esté efectivamente **abierto y corriendo**: a diferencia de la mayoría de las herramientas de esta materia, Docker Desktop necesita su aplicación activa en segundo plano para que el comando `docker` funcione.

3.1 Preparar settings.py para producción

En producción cambian tres cosas respecto de la computadora de desarrollo: la base de datos (PostgreSQL en vez de SQLite), la dirección de Redis y quién sirve los archivos estáticos (el CSS y JS del `/admin/`). Las tres las vamos a leer de variables de entorno, siguiendo el patrón `.env` + `python-dotenv` de la Clase 1: el **mismo código** corre en todos los ambientes y solo cambia la configuración.

Instalen el conector de PostgreSQL y WhiteNoise:

```
pip install "psycopg[binary]" whitenoise
pip freeze > requirements.txt
```

Base de datos. En `taskflow/config/settings.py`, reemplacen el bloque `DATABASES` por:

```
DB_ENGINE = os.getenv("DB_ENGINE", "sqlite")

if DB_ENGINE == "sqlite":
    DATABASES = {
        "default": {
            "ENGINE": "django.db.backends.sqlite3",
            "NAME": BASE_DIR / "db.sqlite3",
        }
    }
else:
    DATABASES = {
        "default": {
            # DB_ENGINE=postgresql  o  DB_ENGINE=mysql
            "ENGINE": f"django.db.backends.{DB_ENGINE}",
            "NAME": os.getenv("DB_NAME"),
            "USER": os.getenv("DB_USER"),
            "PASSWORD": os.getenv("DB_PASSWORD"),
            "HOST": os.getenv("DB_HOST"),
            "PORT": os.getenv("DB_PORT"),
        }
    }
    if DB_ENGINE == "mysql":
        DATABASES["default"]["OPTIONS"] = {"charset": "utf8mb4"}
```

- Si no hay `DB_ENGINE` en el entorno (como en sus computadoras y en GitHub Actions), se usa SQLite, como siempre: no se rompe nada de lo que ya funcionaba.

- Con `DB_ENGINE=postgresql` o `DB_ENGINE=mysql`, arma el `ENGINE` correspondiente (`django.db.backends.postgresql` o `django.db.backends.mysql`) y lee las credenciales del entorno. Son las mismas variables `DB_*` de la clase extra de MySQL, así que ese trabajo sigue sirviendo.
- `psycopg` es el conector entre Django y PostgreSQL (el equivalente de `mysqlclient` para MySQL). La variante `[binary]` viene precompilada, así que no hace falta instalar nada más del sistema.

Redis. No hay que tocar nada: en la Clase 10 ya dejamos `CHANNEL_LAYERS` leyendo `REDIS_URL` del entorno.

Archivos estáticos. Con `DEBUG=False`, Django deja de servir el CSS y JS del `/admin/` y de la Browsable API (en producción eso le corresponde a otro programa). **WhiteNoise** es una librería que le permite a nuestra propia aplicación servirlos de forma eficiente. Agreguen su middleware justo **debajo** de `SecurityMiddleware` en la lista `MIDDLEWARE`:

```
MIDDLEWARE = [
    "corsheaders.middleware.CorsMiddleware",
    "django.middleware.security.SecurityMiddleware",
    "whitenoise.middleware.WhiteNoiseMiddleware",  # nueva
    # ... el resto como estaba
]
```

Y debajo de `STATIC_URL`, agreguen a dónde se juntan los archivos estáticos para producción:

```
STATIC_URL = "static/"
STATIC_ROOT = BASE_DIR / "staticfiles"
```

Orígenes confiables. Al final del archivo:

```
# --- Producción (Clase 12) ---
CSRF_TRUSTED_ORIGINS = [o for o in os.getenv("CSRF_TRUSTED_ORIGINS", "").split(",") if o]
```

Cuando el sitio esté publicado con `https://` en otro dominio, Django exige que ese origen figure en esta lista para aceptar formularios con sesión (por ejemplo, el login del `/admin/`). La comprensión de lista descarta los valores vacíos, así que si la variable no existe queda `[]`.

Agreguen a `.gitignore` la carpeta `staticfiles/` (se genera sola) y la línea `.env.*` (para el archivo de la Parte 3.3).

Antes de seguir, confirmen que no se rompió nada: `python manage.py runserver` sigue funcionando igual que antes (con SQLite) y `pytest` sigue dando `7 passed`.

3.2 Dockerfile

Concepto: contenedores. Un contenedor resuelve el clásico "en mi máquina funciona": empaqueta la aplicación con **su** versión de Python, **sus** librerías y **su** configuración del sistema, y corre igual en cualquier lado.

	Máquina virtual	Contenedor
Qué incluye	Un sistema operativo completo	Solo la aplicación y sus dependencias; comparte el núcleo del sistema del equipo
Tamaño típico	Gigabytes	Decenas o cientos de megabytes
Arranque	Minutos	Segundos
Aislamiento	Muy fuerte	Fuerte, un poco menor

Tres palabras que hay que distinguir:

- **Imagen:** la "receta" ya construida, de solo lectura (lo que genera `docker build`). Se construye en **capas**: cada instrucción del `Dockerfile` agrega una, y Docker reutiliza las capas que no cambiaron. Por eso copiamos `requirements.txt` antes que el resto del código.
- **Contenedor:** una imagen **en ejecución**. De una misma imagen se pueden levantar muchos contenedores iguales (así se escala horizontalmente, Clase 10).
- **Registro:** donde se publican las imágenes para compartirlas (Docker Hub, GitHub Container Registry). De ahí salen `postgres:16` y `redis:7`.

Los contenedores son **efímeros**: todo lo que se escribe adentro se pierde al recrearlos. Lo que tiene que durar va a un **volumen** (como `pgdata`) o a un servicio externo (las bases en la nube).

Práctica.

Un contenedor **Docker** empaqueta la aplicación junto con todo lo que necesita para correr (Python, las dependencias, el código), de forma que funcione igual en cualquier máquina, sin depender de qué esté instalado "a mano" en el servidor de destino. Una **imagen** es la receta ya construida; un **contenedor** es una imagen en ejecución.

Creen el archivo `taskflow/Dockerfile` (sin extensión, ese es el nombre exacto que Docker espera):

```
FROM python:3.12-slim

ENV PYTHONUNBUFFERED=1

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

RUN SECRET_KEY=solo-para-construir python manage.py collectstatic --noinput

CMD ["sh", "-c", "python manage.py migrate --noinput && daphne -b 0.0.0.0 -p 8000
config.asgi:application"]
```

Línea por línea:

- `FROM python:3.12-slim` parte de una imagen base ya armada con Python 3.12 instalado (versión *slim*, más liviana).
- `ENV PYTHONUNBUFFERED=1` hace que los `print` y logs de Python aparezcan al instante en `docker logs`, sin quedar retenidos en memoria.
- `WORKDIR /app` define la carpeta de trabajo **dentro** del contenedor (no confundir con `taskflow/` en la computadora de cada uno: es un sistema de archivos completamente aparte).
- `COPY requirements.txt .` copia primero **solo** ese archivo, a propósito, antes que el resto del código. Si el código cambia pero las dependencias no, Docker reutiliza el paso de instalación ya hecho, y las reconstrucciones son mucho más rápidas.
- `RUN pip install ...` instala las dependencias dentro del contenedor. `--no-cache-dir` evita guardar los instaladores descargados, para que la imagen pese menos.
- `COPY . .` copia el resto del proyecto.
- `RUN ... collectstatic` junta todos los archivos estáticos en `STATIC_ROOT`, para que WhiteNoise los sirva. Le pasamos una `SECRET_KEY` de mentira porque este paso ocurre al **construir** la imagen, cuando todavía no existen las variables de entorno reales (y no queremos que la clave real quede grabada dentro de la imagen).
- `CMD [...]` es el comando que se ejecuta al **arrancar** el contenedor: primero aplica las migraciones pendientes y después levanta **Daphne** escuchando en todas las interfaces (`0.0.0.0`) del puerto 8000. `sh -c "... && ..."` permite encadenar dos comandos: el segundo solo corre si el primero terminó bien.

💡 **Nota del docente**
Usamos **Daphne** (instalado en la Clase 9 con `channels[daphne]`, ya está en `requirements.txt`) en lugar de `runserver`, porque `runserver` es exclusivamente para desarrollo y no está pensado para la carga, la seguridad ni la concurrencia que necesitan los WebSockets en producción.

Creen también `taskflow/.dockerignore`, la lista de lo que **no** se copia a la imagen (funciona igual que `.gitignore`):

```
venv/
.env
.env.*
db.sqlite3
security.log
staticfiles/
.git/
```

```
__pycache__/
```

Es importante por dos motivos: velocidad (no copiar el `venv/` entero, que además no serviría dentro de Linux) y seguridad (que los secretos del `.env` nunca terminen dentro de una imagen que podría publicarse).

3.3 docker-compose para levantar todo junto: app, PostgreSQL y Redis

Creen `taskflow/docker-compose.yml`:

```
services:
  web:
    build: .
    ports: ["8000:8000"]
    env_file: .env.docker
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_started
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: taskflow
      POSTGRES_USER: taskflow
      POSTGRES_PASSWORD: taskflow
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U taskflow -d taskflow"]
      interval: 5s
      retries: 10
  redis:
    image: redis:7

volumes:
  pgdata:
```

`docker compose` orquesta **varios** contenedores relacionados (nuestra app, la base de datos PostgreSQL, Redis) con un solo comando, en vez de arrancarlos manualmente uno por uno como hicimos con Redis en la Clase 10.

- `web` se construye con nuestro `Dockerfile` (`build: .`) y publica el puerto 8000. Lee sus variables de entorno del archivo `.env.docker`.
- `db` y `redis` usan imágenes oficiales ya armadas. Las variables `POSTGRES_*` hacen que PostgreSQL cree la base de datos y el usuario la primera vez que arranca.
- `depends_on` con `condition: service_healthy` hace que `web` espere a que PostgreSQL esté **listo para recibir conexiones** (no solo "arrancado"). Sin eso, el `migrate` del `CMD` podría intentar conectarse cuando la base todavía se está inicializando, y fallar.
- `healthcheck` define cómo se verifica que PostgreSQL está listo: `pg_isready` es una herramienta que viene en la imagen oficial.
- `volumes: pgdata:/var/lib/postgresql/data` persiste los datos de PostgreSQL en disco, para que no se pierdan cada vez que se recrea el contenedor.

Creen el archivo `taskflow/.env.docker`, con la configuración para este ambiente (ya lo agregamos a `.gitignore` en la Parte 3.1 con `.env.*`):

```
DEBUG=False
SECRET_KEY=generen-una-clave-larga-y-distinta-a-la-de-su-.env
ALLOWED_HOSTS=127.0.0.1,localhost
DB_ENGINE=postgresql
DB_NAME=taskflow
DB_USER=taskflow
DB_PASSWORD=taskflow
DB_HOST=db
```

```
DB_PORT=5432
REDIS_URL=redis://redis:6379
```

> 💡 **Nota del docente**
> Noten `DB_HOST=db` y `redis://redis:6379`, no `127.0.0.1`: dentro de la red interna que arma `docker compose`, cada servicio se llama por su **nombre** (`db`, `redis`, tal como quedaron en `docker-compose.yml`), y Docker resuelve ese nombre automáticamente. Para cada contenedor, `127.0.0.1` es **él mismo**, no la computadora ni los otros contenedores.

Levanten todo, parados en `taskflow/`:

```
docker compose up -d --build
docker compose logs -f web
```

`--build` construye la imagen de `web` (la primera vez tarda unos minutos: descarga Python, PostgreSQL y Redis). `-d` deja todo corriendo en segundo plano. `logs -f web` muestra en vivo la salida de nuestra app: van a ver las migraciones aplicándose sobre PostgreSQL y al final `Listening on TCP address 0.0.0.0:8000`. Salgan de los logs con `Ctrl+C` (los contenedores siguen corriendo).

La base de PostgreSQL es **nueva**: creen un usuario adentro del contenedor y prueben:

```
docker compose exec web python manage.py createsuperuser
```

`docker compose exec web <comando>` ejecuta `<comando>` dentro del contenedor `web` que ya está corriendo. Entren a `http://127.0.0.1:8000/admin/` (con su CSS, gracias a WhiteNoise), obtengan un token en `/api/token/`, creen un proyecto y una tarea, y conecten el `escuchar.py` de la Clase 10: toda la aplicación funciona igual que en desarrollo, pero ahora con PostgreSQL, Redis y Daphne, cada uno en su contenedor. Para apagar todo: `docker compose down` (los datos quedan en el volumen `pgdata`).

3.4 Variables de entorno por ambiente

Concepto: paridad entre ambientes. Cuanto más se parezcan desarrollo, staging y producción, menos sorpresas al publicar. La metodología *The Twelve-Factor App* (Clase 8) lo llama **paridad desarrollo/producción**: mismo código, mismos servicios (si producción usa PostgreSQL, conviene probar con PostgreSQL, no con SQLite), y solo cambia la **configuración**, que viaja en variables de entorno. `docker compose` sirve justamente para tener en la propia computadora un entorno casi igual al de producción.

Práctica.

Ya tienen dos archivos de configuración distintos para el mismo código: `.env` (desarrollo, SQLite, `DEBUG=True`) y `.env.docker` (PostgreSQL, Redis en contenedores, `DEBUG=False`). En **staging** y **producción** va a pasar lo mismo, con un `SECRET_KEY` propio de cada ambiente y las credenciales reales de base de datos, nunca los mismos valores que usan en su computadora.

Django trae un chequeo automático de configuración para producción. Pruébenlo con las variables de Docker:

```
docker compose exec web python manage.py check --deploy
```

Muestra una lista de recomendaciones de seguridad (cookies seguras, HSTS, redirección a HTTPS). No son errores que impidan funcionar: son ajustes que se activan cuando el sitio ya está publicado con HTTPS. Léanlas: son el complemento natural de la Clase 8.

3.5 Despliegue en una plataforma cloud gratuita

Con el docente, se despliega el proyecto en una plataforma PaaS (Render o Railway). Los pasos generales, que son los mismos en cualquiera de las dos:

1. Tener la base de datos y el Redis en servicios en la nube gratuitos (Neon o Supabase, y Upstash o Redis Cloud; clase de bases de datos en la nube). **No** usen el PostgreSQL gratuito de Render: se borra a los 30 días.
2. Crear un servicio web conectado al repositorio de GitHub (el mismo que vienen actualizando desde la Clase 1), indicando que se construye con el `Dockerfile`.

3. Cargar las variables de entorno desde el panel de la plataforma: `DEBUG=False`, una `SECRET_KEY` nueva, `ALLOWED_HOSTS` con el dominio que asigna la plataforma (por ejemplo `taskflow-xxxx.onrender.com`), `CSRF_TRUSTED_ORIGINS` con ese mismo dominio y `https://` adelante, `DATABASE_URL` y `REDIS_URL` con los datos de los servicios del paso 1. **Nunca** subiendo el archivo `.env` al repositorio.
4. Disparar el primer despliegue, y crear el superusuario desde la consola (*shell*) que ofrece la plataforma: `python manage.py createsuperuser`.

A partir de este punto, cada `git push` a `main` que pase el pipeline de CI (Parte 2) dispara un nuevo despliegue automático: es el **CD** (Despliegue Continuo), la otra mitad de la sigla "CI/CD".

> 💡 **Nota del docente**
> **Sobre el costo:** tanto Render como Railway ofrecen un **plan gratuito** o de prueba suficiente para el proyecto de esta materia (con límites de horas de uso o de "dormir" el servicio tras inactividad, que no son un problema para fines educativos). Ninguna de las dos es software de código abierto (son servicios comerciales con capa gratuita), y sus condiciones cambian con el tiempo: revísenlas antes de la clase. Si prefieren una alternativa íntegramente de código abierto que corre en su propia infraestructura, pueden autoalojar el mismo `Dockerfile`/`docker-compose.yml` con **Coolify** (`coolify.io`, código abierto, se instala en cualquier servidor propio o VPS) o directamente con `docker compose up -d` sobre cualquier servidor Linux al que tengan acceso. La lógica de contenedores que aprendieron hoy es exactamente la misma; cambia solo quién aloja la máquina.

Parte 4 — Guardar el avance en Git y GitHub, y monitoreo/versionado (30 min)

4.1 Guardar el trabajo de hoy

```
git status
git add .
git commit -m "Agrega tests automatizados, pipeline de CI y configuracion de despliegue con Docker"
git push
```

Antes del `git add`, revisen en `git status` que **no** aparezcan `.env`, `.env.docker`, `staticfiles/` ni `db.sqlite3`. Después del push, vayan a la pestaña **Actions** en GitHub y confirmen la tilde verde.

4.2 Monitoreo y logging en producción

Concepto: observabilidad. Una aplicación en producción tiene que poder responder "¿está funcionando?" y, si no, "¿por qué?", sin que nadie entre al servidor. Eso se llama **observabilidad** y se apoya en tres tipos de datos:

Pilar	Qué es	Ejemplo en TaskFlow
Logs	Registro de eventos puntuales	Login fallido (Clase 8), error al notificar (Clase 10)
Métricas	Números agregados a lo largo del tiempo	Peticiones por minuto, tiempo de respuesta, porcentaje de errores 5xx
Trazas	El recorrido de una petición por todos los servicios	Una petición que pasa por Django, el microservicio de IA y la base

Herramientas habituales, además de los paneles de las plataformas:

- **Chequeo de salud** (*health check*): un endpoint liviano que responde si la aplicación está viva. Ya lo tenemos desde la Clase 1: `/api/health/`. Los servicios de monitoreo lo consultan cada minuto y avisan si deja de responder.
- **Seguimiento de errores:** servicios como Sentry capturan cada excepción con su contexto y avisan al equipo.
- **Alertas:** avisos automáticos cuando una métrica supera un umbral (por ejemplo, más del 5% de respuestas con error).

Práctica.

Repasen el logging de la Clase 8 (`security.log`): en producción, este tipo de logs normalmente se envían a un servicio centralizado (por ejemplo, el panel de logs propio de la plataforma cloud elegida) en lugar de quedar en un archivo local del servidor, para poder buscarlos y recibir alertas sobre errores sin tener que acceder manualmente al servidor. Dentro de un contenedor, además, los archivos locales se **pierden** cada vez que se redespliega: por eso los logs importantes van a la salida estándar (lo que muestra `docker compose logs`) o a un servicio externo.

4.3 Versionado y rollback

Concepto: versionado semántico. La convención más usada para numerar versiones es **SemVer**: `MAYOR.MENOR.PARCHE` (por ejemplo, `1.4.2`).

- **PARCHE** (`1.4.2` → `1.4.3`): se corrigió un error; nada cambia para quien usa la API.
- **MENOR** (`1.4.3` → `1.5.0`): se agregó algo nuevo, compatible con lo anterior (un endpoint o un campo más).
- **MAYOR** (`1.5.0` → `2.0.0`): hay un cambio **incompatible** (se quitó o renombró un campo). Los clientes pueden romperse; en una API REST, suele ir acompañado de un nuevo prefijo (`/api/v2/`).

Concepto: estrategias de despliegue. Publicar una versión nueva también se puede hacer con distintos grados de riesgo:

Estrategia	Cómo funciona	Rollback
Reemplazo gradual (*rolling*)	Las instancias se actualizan de a una	Volver a actualizar, de a una
Azul-verde (*blue-green*)	La versión nueva se levanta completa al lado de la vieja y el tráfico se cambia de golpe	Instantáneo: se vuelve a apuntar a la vieja
Canario (*canary*)	La versión nueva recibe primero un porcentaje chico de usuarios; si va bien, se amplía	Se deja de mandarle tráfico

El "volver a la versión anterior con un clic" de Vercel o Render funciona como un despliegue azul-verde: la versión anterior sigue disponible y solo se cambia cuál recibe el tráfico. Un detalle importante: el código se puede volver atrás fácilmente, pero **las migraciones de base de datos no**. Por eso los cambios en la base se diseñan para ser compatibles con la versión anterior del código (por ejemplo, agregar una columna nueva antes de dejar de usar la vieja).

Práctica.

- Usen **tags** de Git (retomando el documento de Git y GitHub) para marcar versiones desplegadas: `git tag v1.0.0` y después `git push --tags` para subir esa marca a GitHub también.
- Si un despliegue introduce un bug crítico, el **rollback** es volver rápidamente a la versión anterior conocida como estable. La mayoría de las plataformas cloud permiten volver a un despliegue anterior con un clic desde su panel, sin tener que revertir código a mano bajo presión.

Parte 5 — Resumen final y entrega

5.1 Resumen de la materia

En 12 clases construyeron **TaskFlow**: un backend Django/DRF con modelado relacional, CRUD completo, capa de servicios, autenticación JWT, roles y seguridad (OWASP, CORS, rate limiting), tiempo real con Channels y Redis, una capa GraphQL alternativa a REST que respeta las mismas reglas, tests automáticos y un pipeline de CI/CD que despliega en la nube, todo versionado en Git y respaldado en GitHub desde la primera clase.

5.2 Entrega final

1. Repositorio en GitHub (público o compartido con el docente) con el proyecto completo y su historial de commits desde la Clase 1, con el pipeline de CI en verde.
2. URL del despliegue funcionando.
3. Reporte de cobertura de tests (`pytest --cov=core`).
4. Breve documento (1-2 carillas) explicando las decisiones de arquitectura tomadas a lo largo de la materia.

Errores frecuentes

- **`pytest` se corta con `import file mismatch` o `ImportError` en `core/tests.py`**: quedó el archivo `core/tests.py` al lado de la carpeta `core/tests/`. Bórrenlo (Parte 1.3).
- **`Database access not allowed, use the "django_db" mark`**: al test le falta `@pytest.mark.django_db`, o una fixture que use la base no pide `db`.
- **En GitHub Actions: `The SECRET_KEY setting must not be empty`**: falta el bloque `env:` en `ci.yml`.

- **`web` se reinicia en bucle y los logs dicen `connection to server at "db" ... failed`**: falta el `healthcheck` en `db`, o `DB_HOST` no es `db`.
- **El `/admin/` en Docker se ve sin estilos**: falta el middleware de WhiteNoise o el paso `collectstatic` del `Dockerfile`.
- **`Bad Request (400)` al entrar al sitio desplegado**: el dominio no está en `ALLOWED_HOSTS`.
- **`CSRF verification failed` al loguearse en el `/admin/` desplegado**: falta `CSRF_TRUSTED_ORIGINS` con `https://` y el dominio.

Preguntas de repaso

1. Dibujen la pirámide de tests y den un ejemplo de TaskFlow para cada nivel. ¿Por qué hay pocos tests E2E?
2. ¿Qué es el patrón AAA? ¿Qué significan los principios FIRST?
3. Comparen stub, mock y fake, con ejemplos de esta clase.
4. ¿Cómo garantiza `pytest-django` que un test no afecte a otro?
5. ¿Qué diferencia hay entre cobertura de líneas y de ramas?
6. ¿Qué prácticas de trabajo implica la integración continua, más allá de la herramienta?
7. Diferencien entrega continua y despliegue continuo.
8. Definan workflow, job, step, runner y action en GitHub Actions.
9. Comparen contenedores y máquinas virtuales. ¿Qué diferencia hay entre imagen, contenedor y registro?
10. ¿Qué es la paridad desarrollo/producción y cómo ayuda `docker compose`?
11. ¿Cuáles son los tres pilares de la observabilidad?
12. Expliquen SemVer con un ejemplo de cambio en la API de TaskFlow para cada número.
13. Comparen los despliegues gradual, azul-verde y canario. ¿Por qué las migraciones complican el rollback?

Para profundizar

- Documentación de pytest y pytest-django: `docs.pytest.org` y `pytest-django.readthedocs.io`
- Documentación de GitHub Actions: `docs.github.com/es/actions`
- Documentación de Docker: `docs.docker.com/get-started/`
- *The Twelve-Factor App*: `12factor.net/es/`
- Versionado semántico: `semver.org/lang/es/`
- Martin Fowler, *Continuous Integration*: `martinfowler.com/articles/continuousIntegration.html`

Diccionario de términos de esta clase

- **AAA (Arrange, Act, Assert):** estructura de un test: preparar, ejecutar y verificar.
- **Azul-verde (blue-green):** estrategia de despliegue que levanta la versión nueva junto a la vieja y cambia el tráfico de golpe.
- **Canario (canary):** estrategia que expone la versión nueva primero a un grupo chico de usuarios.
- **CD (Despliegue Continuo):** práctica de desplegar automáticamente a producción cada vez que el código pasa las verificaciones de CI.
- **CI (Integración Continua):** práctica de correr automáticamente los tests cada vez que se sube código nuevo.
- **Cobertura de ramas:** medida de cuántos caminos de cada decisión (`if`) fueron probados.
- **Cobertura de tests (coverage):** porcentaje de líneas de código que fueron ejecutadas por al menos un test.
- **Contenedor:** una instancia en ejecución de una imagen Docker, aislada del resto del sistema.
- **Doble de prueba:** reemplazo de una dependencia durante un test (stub, mock, fake).
- **docker compose:** herramienta para definir y ejecutar aplicaciones formadas por varios contenedores relacionados.
- **Dockerfile:** archivo de texto con las instrucciones para construir una imagen Docker.
- **Entrega continua:** automatizar todo hasta dejar una versión lista para publicar, que se publica con una decisión humana.

- **Fixture:** función de `pytest` que prepara datos o configuración y se los entrega a los tests que la piden.
- **Healthcheck:** comando que Docker ejecuta periódicamente para saber si un servicio está listo.
- **Imagen (Docker):** paquete de solo lectura, construido en capas, a partir del cual se crean contenedores.
- **Mock:** reemplazo controlado de una dependencia externa (API, servicio) durante un test.
- **Observabilidad:** capacidad de entender el estado de un sistema a partir de sus logs, métricas y trazas.
- **PaaS (Platform as a Service):** plataforma en la nube que gestiona la infraestructura de despliegue por nosotros (Render, Railway).
- **Pipeline:** secuencia automatizada de pasos (por ejemplo, instalar dependencias → correr tests → desplegar).
- **Pirámide de tests:** modelo que recomienda muchos tests unitarios, bastantes de integración y pocos de punta a punta.
- **Rollback:** volver a una versión anterior conocida como estable, ante un problema en la versión actual.
- **SemVer (versionado semántico):** convención MAYOR.MENOR.PARCHE para numerar versiones.
- **Test de integración:** test que verifica que varias piezas del sistema (por ejemplo, una vista, los permisos y la base de datos) funcionan bien en conjunto.
- **Test de punta a punta (E2E):** test que verifica el sistema completo desde afuera, como lo usaría una persona.
- **Test unitario:** test que verifica una única pieza de código (una función, un método) de forma aislada.
- **Volumen:** almacenamiento de Docker que conserva datos aunque el contenedor se recree.
- **WhiteNoise:** librería que permite a una aplicación Python servir sus propios archivos estáticos en producción.
- **YAML:** formato de texto para archivos de configuración, usado por GitHub Actions y docker compose (no es código Python).